

Technology Stack

Stack Overview

Layer	Technology	Version	Justification
Frontend	React + TypeScript + Vite	React 19.1.1, TS ~5.8.3, Vite 6.5.5	Fast dev/build for SPA, simple routing
UI Framework	SCSS Modules + Sass	Sass 1.89.2	Lightweight styling approach aligned with React
Backend	Node.js + TypeScript	Node.js 20, TS 5.9.3	Familiar ecosystem, fast iteration, scalable
Framework	Express	5.1.0	Lightweight HTTP framework; easy JSON handling
Database	PostgreSQL	16 (Docker image: postgres:16)	Relational integrity for a structured data set
ORM	Prisma	6.19.0	Type-safe DB access, migrations, and queries
Cache	Redis (+ ioredis client)	Redis 7 (redis:7-alpine), ioredis 5.1.0	Simple caching layer for read-heavy endpoints
Message Queue	Not used (MVP)	-	Not required for read-only library MVP
Analytics	Python ETL + pre-rendered SVGs + Plotly.js	Python 3.12, Plotly 5.24.0	Deterministic, cacheable visual outputs
Deployment	Docker + Docker Compose	-	Reproducible local/prod-like environments

Key Technology Decisions

Decision 1: Frontend - React + Vite

Context: Build a responsive SPA for a read-only disease library (cards + search + filters + references/research views) with fast iteration.

Decision: React + Vite.

Rationale: - Fast dev server and build pipeline for frequent UI changes. - Works well for SPA navigation and theming (Dark/Light). - Strong ecosystem for UI components, routing, and testing.

Trade-offs: - Pros: fast iteration, simple SPA architecture, flexible component model. - Cons: no SSR by default; SEO requires basic SPA-friendly practices.

Alternative considered: Next.js (SSR/ISR, stronger SEO) - rejected for MVP due to added complexity not needed for a largely article/library UI.

Decision 2: Backend - Node.js + Express

Context: Provide a simple, stable REST API for diseases/symptoms/risk factors/sources with OpenAPI docs and predictable responses.

Decision: Node.js + Express.

Rationale: - Lightweight and familiar for JSON APIs. - Fits well with a React SPA and Docker-based local environment.

Trade-offs: - Pros: fast iteration, minimal boilerplate, strong middleware ecosystem. - Cons: requires separate Python path for analytics scripts (kept decoupled by design).

Alternative considered: Django (Python) - rejected for MVP because it is a heavier stack and slower iteration with an existing JS frontend.

Decision 3: Analytics & Visualization - Python ETL + SVG embeds

Context: Generate simple, trustworthy visualizations without adding complex BI runtime dependencies.

Decision: Python ETL + pre-rendered SVG embeds.

Rationale: - Deterministic outputs suitable for caching and static delivery. - Avoids licensing/vendor lock-in and minimizes runtime complexity. - Keeps analytics development independent from the web runtime.

Alternative considered: Power BI/Fabric or live JS charting only - rejected for licensing/vendor lock-in and increased runtime complexity; interactive JS charts can be added later if needed.

Decision 4: State management - Redux Toolkit (current) vs local state (initial idea)

Context: MVP initially expected mostly local/query state for cards/search/filters.

Decision: Redux Toolkit is present and used for data fetching and shared state in the current codebase (thunks + slices; RTK Query scaffold).

Rationale: - Centralizes API data lifecycle (loading/error/results) for multiple views. - Easier to extend for caching, saved views, and more complex UI flows.

Alternative considered: No Redux initially - acceptable for a very small UI, but the project already includes Redux Toolkit to support growth.

Decision 5: Database modeling - normalized schema (implemented)

Context: The MVP text mentions a single diseases table to ship fast.

Decision: The implemented schema is normalized: Disease, Symptom, RiskFactor, join tables (DiseaseSymptom, DiseaseRiskFactor), Source, and translation tables per locale.

Rationale: - Supports primary/secondary symptom modeling and risk factor direction. - Supports localization (EN/RU) at the data layer. - Improves queryability and data integrity compared to a single denormalized table.

Development Tools

Tool	Purpose	Notes
IDE	VS Code	ESLint/Prettier integration, TypeScri...
Version Control	Git	MVP workflow; hooks enforce formattin...
Package Manager	npm	Frontend/backend dependency management.

Linting & Formatting	ESLint + Prettier (+ Stylelint for SCSS)	Auto-fix enabled; lint-staged + Husky...
Testing (FE)	Jest + React Testing Library; Playwri...	Unit/component + end-to-end coverage.
Testing (BE)	Jest + Supertest	Controller/service/repository tests.
Build tooling (BE)	SWC	Fast TypeScript compilation to build/.
DB Tooling	Prisma CLI	Migrations, client generation, seeding.
Documentation	OpenAPI (express-jsdoc-swagger), Stor...	API docs at /api-docs; component docs...
Containers	Docker + Docker Compose	Runs API + PostgreSQL + Redis locally.

External Services & APIs

Service	Purpose	Pricing Model
Sentry	Error monitoring for frontend and bac...	Free tier / paid plans
WHO Data	Official datasets for indicators and ...	Free (open data; license-dependent)
Eurostat	Official EU statistics (indicators by...	Free (open data; license-dependent)
CDC WONDER	Official public-health datasets	Free (open data; terms apply)